



Aqueduct Materials (aqueducts)

Los acueductos se construyen usando alguno de los N materiales de construcción romanos. El i -ésimo material tiene una resistencia S_i y un precio P_i .

Decimos que el material i -ésimo *domina* el material j -ésimo si y sólo si:

- es más resistente ($S_i > S_j$), y
- es más barato ($P_i < P_j$).

El Senado quiere reservar **un** material de manera que sea de uso exclusivo en Roma, para conseguir que la ciudad tenga un aspecto único. Los restantes $N - 1$ materiales estarán *disponibles* para el resto del imperio.

Un material es *óptimo* para el resto del imperio si

- está disponible, y
- no está dominado por otro material disponible.

Para cada k , suponiendo que el material reservado para Roma es k , cuenta la cantidad C_k de materiales óptimos para el resto del imperio.

Implementación

Deberás entregar un único archivo `.cpp`.



Entre los archivos adjuntos de este ejercicio encontrarás en `aqueducts.cpp` una plantilla de ejemplo de implementación.

Deberás implementar la siguiente función:

C++

```
vector<int> count(int N, vector<int> S, vector<int> P)
```

- El número entero N representa el número de materiales.
- El vector S , indexado de 0 a $N - 1$, contiene los valores $S_0, S_1, \dots, S_{N-1}$.
- El vector P , indexado de 0 a $N - 1$, contiene los valores $P_0, P_1, \dots, P_{N-1}$.
- La función debe devolver un vector C , conteniendo los valores $C_0, C_1, \dots, C_{N-1}$ donde C_k es el número de materiales óptimos restantes para el imperio si el material reservado para Roma es el k .

Sample Grader

Entre los archivos adjuntos de este ejercicio encontrarás una versión simplificada del evaluador («grader») que puedes usar para comprobar tus soluciones de manera local. El evaluador lee la entrada desde `stdin`, llama a la función `count` y escribe en `stdout`, usando el siguiente formato:

La entrada está compuesta por hasta $N + 1$ líneas, con el siguiente contenido:

- Línea 1: el entero N .
- Línea $2 + i$ ($0 \leq i < N$): los enteros S_i y P_i .

La salida consiste en una única línea que contiene los valores devueltos por la función `count`.

Restricciones

- $2 \leq N \leq 300\,000$.

- $1 \leq S_i, P_i \leq 1\,000\,000$.
- Para todo $0 \leq i < j \leq N - 1$, tenemos que $S_i \neq S_j$ y $P_i \neq P_j$.

Puntuación

Tu programa se ejecutará con diversas casos de prueba, agrupadas por sub-ejercicios. Para obtener la puntuación de cada sub-ejercicio tu programa debe resolver correctamente todos sus casos de prueba.

- **Subtask 0 [0 puntos]**: Casos de prueba de ejemplo.
- **Subtask 1 [13 puntos]**: $N \leq 100$.
- **Subtask 2 [22 puntos]**: $N \leq 5000$.
- **Subtask 3 [28 puntos]**: $|S_i - P_i| \leq 2$.
- **Subtask 4 [37 puntos]**: Sin restricciones adicionales.

Ejemplos de entrada/salida

stdin	stdout
3 10 20 30 10 20 30	1 2 1
9 10 11 4 6 11 13 14 7 1 4 6 10 9 2 12 12 3 5	2 2 2 3 2 2 4 2 2

Explicación

Explicación del primer ejemplo.

- Si $k = 0$, el material 1 es óptimo.
- Si $k = 1$, los materiales 0 y 2 son óptimos.
- Si $k = 2$, el material 1 es óptimo.

Explicación del segundo ejemplo.

- Si $k = 0$, los materiales 3 y 6 son óptimos.
- Si $k = 1$, los materiales 3 y 6 son óptimos.
- Si $k = 2$, los materiales 3 y 6 son óptimos.
- Si $k = 3$, los materiales 0, 6 y 7 son óptimos.
- Si $k = 4$, los materiales 3 y 6 son óptimos.
- Si $k = 5$, los materiales 3 y 6 son óptimos.
- Si $k = 6$, los materiales 1, 3, 4 y 8 son óptimos.
- Si $k = 7$, los materiales 3 y 6 son óptimos.
- Si $k = 8$, los materiales 3 y 6 son óptimos.